

Tested Features

Antivirus

- Pathfinding
 - Around obstacles
- Killing player

Player

- Movement
 - No moving into walls, or out of bounds
- Collecting data
- Collecting source code
- Damaging by firewall
 - Losing (data < 0) from firewall damage
- Exiting to win
 - Cannot exit when data left over

In-Game Collectables

- Can be collected by the player
- No interaction with Antivirus entity
- Decryption Key
 - Can unlock door
- Source code spawning
 - Source code expiry
- Firewall spread
 - Blocked by walls, other entities
 - Stopped when frozen
- Freeze token collection
 - Causes firewall spread freezing

Board Generation

- 6 Data Packets, 10 Firewalls, 1 Antivirus, and 1 Player are present.
- No random walls in room interiors, buffer zones, or doorways.
- The server room door is initially locked.

UI Menu and Rendering

- Title menu shows at game start.
 - Animated background frames

- Options menu shows on “Options” button click
 - Title menu shows on “Back” button click
 - Volume can be set, parametrized testing with values from 0, 25, 75, 100
- Starting game on “Start Mission” button click
 - Tutorial appears and dismissible with “Got it” button
- In game, pause menu activated with ESC key press or pause icon button click
 - Pause menu dismissed on “Resume” button click
 - Title menu shows on “Exit” button click
- Win and lose screen correctly showing info
- Animated entity (player, antivirus)
 - Correct animated frames used while heading left/right
 - Idle frame used while not moving
 - Fallback to idle if animated frames missing
- Asset and audio management
 - Caching assets on a map
 - Playing audio on set volume
 - Asset fallback on missing files, null stream, exception

Component Interactions

Our codebase is logically split into three parts: the board (game logic), the board generation system and the UI (rendering logic).

When the W/A/S/D keys are pressed, the player should move

GameMenuTest#playerMovement

Simulates keyboard input in the UI, and verifies that the board has received the movement events and the player moves at the next tick.

When the game is paused, the player should not move

GameMenuTest#playerMovementDuringPause

Pauses the game with the ESCAPE key and then presses movement keys. Then it verifies that the board has received the pause state and player does not move.

When the player takes damage in game, the screen flashes

GameMenuTest#damageFlash

Simulates player damage via the board’s player entity and verifies that the UI successfully creates a damage flash effect.

The app can launch to the title screen, and a game can be started. When it is, a board should be generated and the game UI displayed

VirusBreachTest#canStartGame

Launches the game, and presses the “start mission” button, and ensures that the UI is initialized with the generated board.

The HUD should update based on the board state

HudTest#updateShowsTimeAndScore

Generates a board and sets up the UI for rendering it. After the board updates and the player earns score, ensures that the UI clock has been updated and the score display has also been updated.

The HUD should show a notification when the exit is unlocked

HudTest#updateShowsExitText

Generates a board with all data collected and synchronizes it to the UI. Then it ensures that a notification pop up is displayed for the exit condition.

When data is collected the HUD should be updated

GameMenuTest#entityRemovedData

Generates a board with a player and data, and sets up the UI for rendering it. After the data is removed from the board, ensures that the UI display of collected data has also been updated, and that the node for the collected data has been removed from the screen.

Ensuring Test Quality

While implementing the game, we separated the game (board) logic from the rendering of the game, which allowed us to have all game logic run asynchronously on a separate thread. This practice has made testing easier for most of the tests, since we don’t need to initialize JavaFX to simulate the game, except for test cases that target UI components. You can particularly see this in the tests in the board package (and subpackages), which do not need to interface with JavaFX in any way. This allows them to run rapidly and complete in milliseconds.

In each test for game logic, `TestHelper` is used to create an enclosed board that is simple for testing and is independent from other tests. Each functional test targets a behaviour of the game and assert observable outcomes as a result of the behaviour.

We copied the same package layout for the tests as the main project layout. This allowed us to utilize package-private access modifiers to let tests assert on internal object state,

without exposing it to everything else. Tests are generally named the same as their base class name with “Test” appended, so that they are easy to locate.

For UI tests, we used the TestFX library that gives us a `robot` to click and read UI components. Since the testing thread and the JavaFX application thread run asynchronously, the function `waitForFxEvents()` is used to wait for events after an action (e.g. clicking) before asserting. TestFX also provides an assert function `verifyThat` to match visible nodes to the expected UI state. Explicit setup and cleanup are added to prevent shared JavaFX state from affecting other tests.

Code Coverage

100% line and branch coverage on:

- root: VirusBreach, GameOptions
- math: Direction, Position
- board: Room, ServerRoom, StorageRoom, BoardObserver
- board.tile: Exit, Entrance, LockedDoor, Floor, OpenDoor, Wall, TileType
- board.entity: Antivirus, Player, Firewall, Entity, SourceCode, DecryptionKey, Collectable, FreezeToken, Data
- ui: AnimatedNode, RenderNode, AudioManager, ResourceManager, RenderNodeRegistry, TutorialOverlay, HUD
- ui.menu: GameMenu, WinMenu, GameOverMenu, TitleMenu, OptionsMenu, PauseMenu, Menu

Almost full coverage on:

- Root: BoardGenerator (100% line coverage, 90% branch coverage). The remaining missed branches are due to the randomness of board generation and extra sanity checks inside `placeWall()`. In rare scenarios, the test may not execute certain branches due to the outputs of the random number generator. Additionally, there are extra bounds checks to ensure that we never generate invalid boards, that do not get tripped by normal inputs to the board generation system. For 100% coverage, tests for these irrelevant inputs are needed to call this internal method, which does not add meaningful testing value.
- Board: Board class has 99% line coverage (1 miss) and 94% branch coverage (1 miss) due to a special case required when running tests. This class shuts down the runtime of our UI library JavaFX when an exception occurs. Some of our tests, however, intentionally cause exceptions to ensure that we can fail properly and

gracefully. Unfortunately, when running tests, we do not want to shut down the entire testing environment, as only one JavaFX runtime can be started per JVM execution. Therefore, we added an if statement to detect whether the game is under testing, and if so, we skip the shutdown of the JavaFX runtime causing the single missed line and branch.

No Coverage:

- Main: This is the normal entrypoint for the game that simply consists of a main() method that launches the JavaFX runtime and application. Due to limitations with JavaFX and JUnit, we can't test launching through this class. This class must be manually tested by launching the game.

Findings

Some of our production code was changed during the testing phase to fix revealed bugs, improve code quality, and assist with testing.

Production changes made to fix bugs:

- Firewall spread freezing state being stored globally, and able to leak out into the next game
 - Fixed by moving the freezing state to be a field on the board.
- Handling the error when an animated entity is missing a sprite
 - Fixed by taking generating a backup/fallback asset when missing.

Production changes made to improve code quality:

- Removed dead code in decryption key unlocking
 - A field tracking the decryption key pickup was set and never used on the Player entity

Production changes made to improve testability:

- Added package-private getters and setters to initially internally private states that were needed to be asserted
- Made some methods package-private instead of fully private
- As mentioned in test coverage, the JavaFX runtime shutdown is conditioned to not run during testing
- Changes to board entity management to not require a full game tick to process entity changes. This allowed for board testing code to be streamlined significantly

as entities could be added and removed from the board at will, and have the changes be reflected immediately.